HILL CLIMBING

```
import random

N = 4

# Calculate the number of attacking queen pairs
def calculate_cost(state):
    cost = 0

    for i in range(N):
        for j in range(i + 1, N):
            if state[i] == state[j] or abs(state[i] - state[j]) == abs(i -
j):
                cost += 1

    return cost


# Display the chessboard
def display_board(state):
    for row in range(N):
        for col in range(N):
            if state[col] == row:
                print("Q", end=" ")
            else:
                print("-", end=" ")
        print()


# Generate neighboring states
def get_neighbors(state):
    neighbors = []

    for col in range(N):
        for row in range(N):
```

```
            if row != state[col]:
                neighbor = state.copy()
                neighbor[col] = row
                neighbors.append(neighbor)

    return neighbors


# Hill Climbing algorithm with random restart
def hill_climbing():
    while True:
        current = [random.randint(0, N - 1) for _ in range(N)]

        print("\nInitial State:", current)

        while True:
            current_cost = calculate_cost(current)

            print("Current State:", current,
                  "Cost:", current_cost)

            if current_cost == 0:
                print("\nGoal State Reached!")
                display_board(current)
                return

            neighbors = get_neighbors(current)

            print("Neighbor Costs:")
            for neighbor in neighbors:
                print(neighbor, "Cost:",
                      calculate_cost(neighbor))

            best_neighbor = min(neighbors,
                                key=calculate_cost)

            best_cost = calculate_cost(best_neighbor)

            if best_cost >= current_cost:
                print("Local minimum reached. Restarting...")
```

```
                break

            current = best_neighbor


hill_climbing()
```

```
    Initial State: [2, 2, 3, 0]
    Current State: [2, 2, 3, 0] Cost: 3
    Neighbor Costs:
    [0, 2, 3, 0] Cost: 3
    [1, 2, 3, 0] Cost: 4
    [3, 2, 3, 0] Cost: 5
    [2, 0, 3, 0] Cost: 1
    [2, 1, 3, 0] Cost: 1
    [2, 3, 3, 0] Cost: 2
    [2, 2, 0, 0] Cost: 4
    [2, 2, 1, 0] Cost: 4
    [2, 2, 2, 0] Cost: 4
    [2, 2, 3, 1] Cost: 2
    [2, 2, 3, 2] Cost: 5
    [2, 2, 3, 3] Cost: 3
    Current State: [2, 0, 3, 0] Cost: 1
    Neighbor Costs:
    [0, 0, 3, 0] Cost: 3
    [1, 0, 3, 0] Cost: 3
    [3, 0, 3, 0] Cost: 3
    [2, 1, 3, 0] Cost: 1
    [2, 2, 3, 0] Cost: 3
    [2, 3, 3, 0] Cost: 2
    [2, 0, 0, 0] Cost: 4
    [2, 0, 1, 0] Cost: 3
    [2, 0, 2, 0] Cost: 2
    [2, 0, 3, 1] Cost: 0
    [2, 0, 3, 2] Cost: 3
    [2, 0, 3, 3] Cost: 1
    Current State: [2, 0, 3, 1] Cost: 0

    Goal State Reached!
    - Q - -
    - - - Q
    Q - - -
    - - Q -
```

---

8 QUEENS

```
import random
import math

N = 8

# Calculate the number of attacking queen pairs
def calculate_cost(state):
    cost = 0

    for i in range(N):
        for j in range(i + 1, N):
            if state[i] == state[j] or abs(state[i] - state[j]) == abs(i -
j):
                cost += 1

    return cost


# Display the chessboard
def display_board(state):
    for row in range(N):
        for col in range(N):
            if state[col] == row:
                print("Q", end=" ")
            else:
                print("-", end=" ")
        print()


# Simulated Annealing algorithm
def simulated_annealing():
    while True:
        # Generate a random initial state
        current = [random.randint(0, N - 1) for _ in range(N)]
        current_cost = calculate_cost(current)
```

```
        temperature = 100.0
        cooling_rate = 0.99

        while temperature > 0.001:
            if current_cost == 0:
                print("Solution Found!")
                print("State:", current)
                print("Cost:", current_cost)
                display_board(current)
                return

            # Generate a neighboring state
            neighbor = current.copy()
            col = random.randint(0, N - 1)
            neighbor[col] = random.randint(0, N - 1)

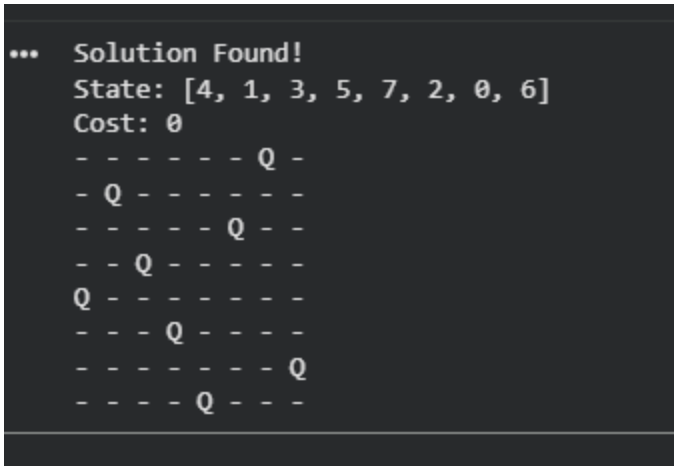
            neighbor_cost = calculate_cost(neighbor)
            delta = neighbor_cost - current_cost

            # Accept better states or sometimes worse states
            if delta < 0:
                current = neighbor
                current_cost = neighbor_cost
            elif random.random() < math.exp(-delta / temperature):
                current = neighbor
                current_cost = neighbor_cost

            # Reduce the temperature
            temperature *= cooling_rate

        # Restart if no solution is found


simulated_annealing()
```



—----------THE END —----------